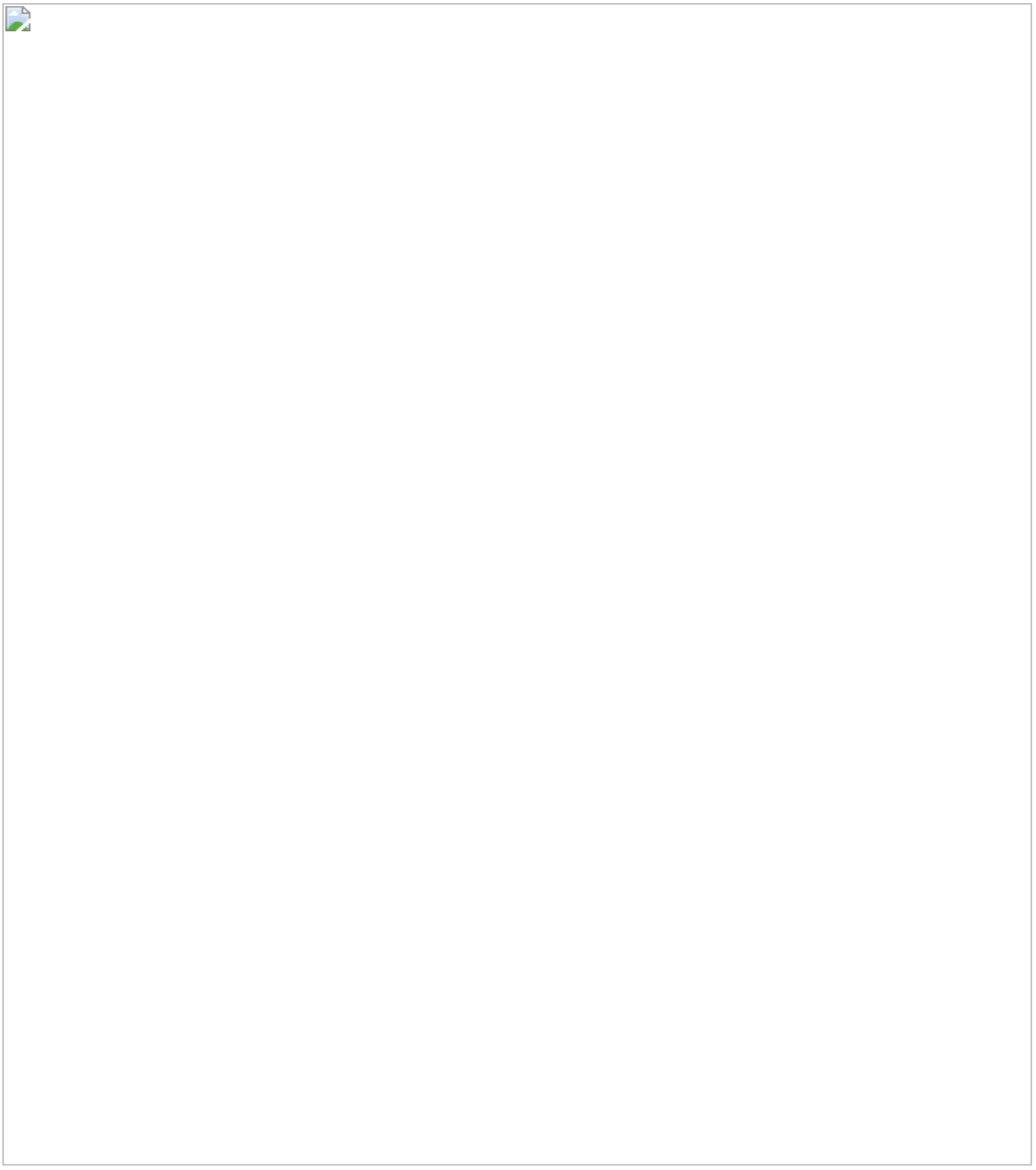
Claude

claude.ai/share/750d2f09-a903-4e94-ada5-101dae1202ec



This is a copy of a chat between Claude and Hearaman. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

How to change the position on BackstageHeaderTabs? Now they are appearing next to BackstageHeader. I want to change position of it.

9:03 AM

Looking at your `themes.ts`, the `BackstageHeaderTabs` override only has `tabsWrapper` and `defaultTab` slots. To reposition the tabs, you'll need to add layout/position CSS to the `tabsWrapper` slot.

Here are the most common fixes depending on what you want:

Option 1 — Stick tabs below the header (most common fix for "appearing next to" issue):

ts

```
BackstageHeaderTabs: {
  styleOverrides: {
    tabsWrapper: {
      backgroundColor: t.bgPaper,
      borderBottom: `1px solid ${t.borderSubtle}`,
      width: '100%',        // ← force full-width block
      display: 'block',     // ← break out of any flex row
      order: 1,             // ← push after header in flex stacking
    },
    defaultTab: {
      textTransform: 'none',
      fontWeight: t.weightBold,
      color: t.textSecondary,
    },
  },
},
```

Option 2 — Make tabs sticky so they follow on scroll:

ts

```
tabsWrapper: {
  backgroundColor: t.bgPaper,
  borderBottom: `1px solid ${t.borderSubtle}`,
  position: 'sticky',
  top: 0,
  zIndex: 1100,
  width: '100%',
},
```

Option 3 — Offset tabs below a fixed-height header:

ts

```
tabsWrapper: {
  backgroundColor: t.bgPaper,
  borderBottom: `1px solid ${t.borderSubtle}`,
  marginTop: '64px', // match your header height
},
```

The root cause of "appearing next to the header" is usually that the parent container uses `display: flex` with `flex-direction: row`, so adding `width: '100%'` + `display: 'block'` (Option 1) is usually the right first thing to try. If that doesn't fully fix it, also check if your `BackstageHeader` header override needs `flexShrink: 0` or `alignSelf: 'flex-start'` to stop it from stretching alongside the tabs.

In the catalog details page, I want to change the default content coming that is entity name and context dropdown. How can i add my own content there in header

9:08 AM

The catalog entity page header (entity name + context dropdown) is controlled in your app's `EntityPage.tsx`, not in the theme. Here's how to customize it:

Locate the file

```
packages/app/src/components/catalog/EntityPage.tsx
```

How it works

The header content comes from `EntityLayout`'s props. By default it looks like:

tsx

```
<EntityLayout>
  <EntityLayout.Route path="/" title="Overview">
    ...
  </EntityLayout.Route>
</EntityLayout>
```

`EntityLayout` internally reads `useEntity()` and renders the name + context menu automatically.

Option 1 — Override the title with custom content

tsx

```
import { useEntity } from '@backstage/plugin-catalog-react';

const MyEntityPage = () => {
  const { entity } = useEntity();

  return (
    <EntityLayout
      title={
        <span>
          {entity.metadata.name}
          <MyCustomBadge /> {/* your own component */}
        </span>
      }
    >
      ...
    </EntityLayout>
  );
};
```

Option 2 — Add extra header actions (replaces context dropdown area)

tsx

```
<EntityLayout
  UNSTABLE_extraContextMenuItems={[
    {
      title: 'My Action',
      Icon: MyIcon,
      onClick: () => doSomething(),
    },
  ]}
>
```

Option 3 — Fully custom header using `Header` directly

If you want complete control, bypass `EntityLayout`'s header and build your own:

tsx

```
import { Header, Page, Content } from '@backstage/core-components';
import { useEntity } from '@backstage/plugin-catalog-react';

const CustomEntityPage = () => {
  const { entity } = useEntity();

  return (
    <Page themeId="service">
      <Header
        title={entity.metadata.name}
        subtitle={<MyCustomSubtitle entity={entity} />}
      >
        {/* anything you want here — buttons, dropdowns, chips */}
        <MyOwnershipBadge entity={entity} />
        <MyRunPipelineButton entity={entity} />
      </Header>
      <Content>
        {/* your tabs/routes manually */}
      </Content>
    </Page>
  );
};
```

Which to use?

Goal	Approach
Change the title text/style	Option 1
Add items to the ⋮ context menu	Option 2
Fully replace header with custom layout	Option 3

Option 3 gives you the most flexibility but you lose the built-in context menu and routing that `EntityLayout` provides out of the box. Option 1 is the least invasive.

Remember, I am using latest backstage version 1.49.0 and its new front-end system

9:09 AM

Good call — the old `EntityPage.tsx` + `EntityLayout` approach doesn't apply in the new frontend system. Everything is extension-based now.

How it works in the new frontend system

The catalog entity page is composed of extensions. You customize it by **overriding extensions** in your app, not by editing a component file directly.

Step 1 — Find the entity header extension

The catalog plugin exposes the entity page header as an extension. You can inspect what's available:

ts

```
import { catalogPlugin } from '@backstage/plugin-catalog';

// In your browser console or by inspecting:
catalogPlugin.extensions; // lists all exposed extensions
```

Step 2 — Create a custom header extension override in your app

In `packages/app/src/App.tsx` (or your extension file):

ts

```
import {
  createExtension,
  createExtensionOverrides,
  coreExtensionData,
} from '@backstage/frontend-plugin-api';
import { useEntity } from '@backstage/plugin-catalog-react';
import { Header } from '@backstage/core-components';

const CustomEntityHeader = createExtension({
  // Match the exact ID of the catalog's header extension
  name: 'catalog/entity-page-header',
  attachTo: { id: 'page:catalog/entity', input: 'header' },
  output: [coreExtensionData.reactElement],
  factory({ inputs }) {
    return [
      coreExtensionData.reactElement(
        <CustomHeaderComponent />
      ),
    ];
  },
});

export default createExtensionOverrides({
  extensions: [CustomEntityHeader],
});
```

Step 3 — Register it in `app-config.yaml` or `App.tsx`

ts

```
// App.tsx
import { createApp } from '@backstage/frontend-app-api';

const app = createApp({
  features: [
    // your override feature
    myCustomEntityHeaderOverride,
  ],
});
```

Important caveat

The **exact extension IDs** for the catalog entity header depend on what `@backstage/plugin-catalog` exposes in 1.49.0. I'd recommend:

bash

```
# Check the actual extension IDs being registered
yarn backstage-cli package start --inspect
```

Or look at the catalog plugin source for the exact `createExtension` definitions and their `name`/`attachTo` values so your override targets the right extension ID.

The new system's flexibility is that once you have the right extension ID, you can replace any piece of the page without touching the plugin's internals.

> Turn on web search in Search and tools menu. Otherwise, links provided may not be accurate or up to date.

How can i pass values to CustomHeaderComponent based on the page.

9:13 AM

In the new frontend system, since `factory` just returns a React element, you simply **use hooks inside the component itself**. The factory function is not the right place to fetch data — keep it thin.

The right pattern

tsx

```tsx
import { useEntity } from '@backstage/plugin-catalog-react';
import { useRouteRef } from '@backstage/frontend-plugin-api';

// ——— Your custom header component ——————————————————————
const CustomHeaderComponent = () => {
  // 1. Get the current entity from context
  const { entity } = useEntity();

  // 2. Derive what you need from entity
  const name = entity.metadata.name;
  const kind = entity.kind;
  const namespace = entity.metadata.namespace ?? 'default';
  const description = entity.metadata.description;
  const owner = entity.spec?.owner as string | undefined;
  const lifecycle = entity.spec?.lifecycle as string | undefined;
  const annotations = entity.metadata.annotations ?? {};
  const labels = entity.metadata.labels ?? {};

  return (
    <Header
      title={name}
      subtitle={description}
      type={kind}
      typeLink="/catalog"
    >
      {/* Render anything conditional based on kind */}
      {kind === 'Component' && (
        <MyOwnerChip owner={owner} />
      )}
      {kind === 'API' && (
        <MyApiTypeBadge entity={entity} />
      )}

      {/* Render based on annotations */}
      {annotations['github.com/project-slug'] && (
        <MyGithubButton slug={annotations['github.com/project-slug']} />
      )}

      {/* Render based on lifecycle */}
      {lifecycle === 'production' && (
        <MyProductionBadge />
      )}
    </Header>
  );
};
```

Handling different entity kinds (like EntityPage.tsx used to)

tsx

```
const CustomHeaderComponent = () => {
  const { entity } = useEntity();

  // Render a different header per kind
  switch (entity.kind.toLowerCase()) {
    case 'component':
      return <ComponentHeader entity={entity} />;
    case 'api':
      return <ApiHeader entity={entity} />;
    case 'system':
      return <SystemHeader entity={entity} />;
    default:
      return <DefaultHeader entity={entity} />;
  }
};

// Each header gets the full entity and picks what it needs
const ComponentHeader = ({ entity }: { entity: Entity }) => (
  <Header
    title={entity.metadata.name}
    subtitle={entity.spec?.lifecycle as string}
    type={entity.spec?.type as string}
  />
);
```

If you need catalog relations (e.g. owner details)

tsx

```
import {
  useEntity,
  useRelatedEntities,
} from '@backstage/plugin-catalog-react';

const CustomHeaderComponent = () => {
  const { entity } = useEntity();

  const { entities: ownerEntities } = useRelatedEntities(entity, {
    type: 'ownedBy',
  });

  const ownerName = ownerEntities?.[0]?.metadata.name ?? 'Unknown';

  return (
    <Header title={entity.metadata.name} subtitle={`Owned by ${ownerName}`} />
  );
};
```

Key points

What you need	How to get it
Entity name, kind, namespace	`useEntity()`

What you need	How to get it
Annotations / labels	`entity.metadata.annotations`
Relations (owner, subcomponent of)	`useRelatedEntities()`
Current route params	`useRouteRef()` or `useParams()`
Different UI per kind	`switch` on `entity.kind`

The factory stays completely thin — all the data fetching and conditional logic lives inside the component using hooks, exactly like any normal React component.

How can i create `BackstageHeaderTabs` by my own with new look and feel, instead of depending on backstage default. I should able to generate it, if i am correct based on EntityContentBluePrint

9:18 AM

Yes, exactly right! `EntityContentBlueprint` is what generates each tab. Here's how to build the full custom tab system:

How it works

```
EntityContentBlueprint  →  defines each tab (title + path + content)
Custom Layout Extension →  collects all tabs, renders YOUR tab bar
```

Step 1 — Define your tabs with `EntityContentBlueprint`

tsx

```
// src/extensions/entityTabs.tsx
import { EntityContentBlueprint } from '@backstage/plugin-catalog-react/alpha';

export const entityOverviewTab = EntityContentBlueprint.make({
  name: 'overview',
  params: {
    title: 'Overview',
    path: '/',
    filter: 'kind:component',  // optional — show only for certain kinds
    content: async () => {
      const { OverviewPage } = await import('../pages/OverviewPage');
      return <OverviewPage />;
    },
  },
});

export const entityCiCdTab = EntityContentBlueprint.make({
  name: 'cicd',
  params: {
    title: 'CI/CD',
    path: '/cicd',
    filter: 'kind:component,spec.type:service',
    content: async () => {
      const { CiCdPage } = await import('../pages/CiCdPage');
      return <CiCdPage />;
    },
  },
});
```

Step 2 — Build your custom Tab Bar component

tsx

```
// src/components/CustomEntityTabs.tsx
import React, { useState } from 'react';
import { useNavigate, useLocation } from 'react-router-dom';
import { useEntity } from '@backstage/plugin-catalog-react';

type Tab = {
  title: string;
  path: string;
  content: React.ReactNode;
};

type CustomEntityTabsProps = {
  tabs: Tab[];
};

export const CustomEntityTabs = ({ tabs }: CustomEntityTabsProps) => {
  const location = useLocation();
  const navigate = useNavigate();
  const { entity } = useEntity();

  const activeTab = tabs.find(tab =>
    location.pathname.endsWith(tab.path)
  ) ?? tabs[0];

  return (
    <div>
      {/* ——— Your custom tab bar ——————————————— */}
      <div
        style={{
          display: 'flex',
          gap: 4,
          padding: '0 24px',
          borderBottom: `2px solid ${YOUR_TOKEN}`,
          backgroundColor: YOUR_BG_TOKEN,
        }}
      >
        {tabs.map(tab => {
          const isActive = tab.path === activeTab.path;
          return (
            <button
              key={tab.path}
              onClick={() => navigate(tab.path)}
              style={{
                padding: '10px 20px',
                border: 'none',
                borderBottom: isActive
                  ? `3px solid ${YOUR_PRIMARY_TOKEN}`
                  : '3px solid transparent',
                background: 'transparent',
                fontWeight: isActive ? 700 : 400,
                color: isActive ? YOUR_PRIMARY : YOUR_TEXT_SECONDARY,
                cursor: 'pointer',
                borderRadius: '6px 6px 0 0',
                transition: 'all 140ms ease',
              }}
            >
```

```
              {tab.title}
            </button>
          );
        })}
      </div>

      {/* ─── Active tab content ──────────────────── */}
      <div style={{ padding: 24 }}>
        {activeTab.content}
      </div>
    </div>
  );
};
```

Step 3 — Create a custom Entity Page Layout extension

This is the key part — you collect all `EntityContentBlueprint` inputs and render your own tab bar:

tsx

```
// src/extensions/CustomEntityLayout.tsx
import {
  createExtension,
  coreExtensionData,
  createExtensionInput,
} from '@backstage/frontend-plugin-api';
import { entityContentData } from '@backstage/plugin-catalog-react/alpha';

export const CustomEntityLayout = createExtension({
  name: 'custom-entity-layout',
  attachTo: { id: 'page:catalog/entity', input: 'content' },

  // Collect all tabs registered via EntityContentBlueprint
  inputs: {
    tabs: createExtensionInput([
      entityContentData.title,
      entityContentData.path,
      entityContentData.content,
    ]),
  },

  output: [coreExtensionData.reactElement],

  factory({ inputs }) {
    // Map blueprint inputs into tab shape
    const tabs = inputs.tabs.map(tab => ({
      title: tab.get(entityContentData.title),
      path:  tab.get(entityContentData.path),
      content: tab.get(entityContentData.content),
    }));

    return [
      coreExtensionData.reactElement(
        <CustomEntityTabs tabs={tabs} />
      ),
    ];
  },
});
```

Step 4 — Wire everything in `App.tsx`

ts

```
import { createApp } from '@backstage/frontend-app-api';

const app = createApp({
  features: [
    // Your custom layout replaces the default
    createExtensionOverrides({
      extensions: [
        CustomEntityLayout,
        entityOverviewTab,
        entityCiCdTab,
      ],
    }),
  ],
});
```

Summary flow

```
App registers EntityContentBlueprint tabs (overview, cicd, ...)
        ↓
CustomEntityLayout collects them via inputs.tabs
        ↓
Maps them → { title, path, content }[]
        ↓
CustomEntityTabs renders YOUR tab bar UI with those tabs
        ↓
useLocation() tracks active tab, navigate() handles routing
```

The key insight is the `inputs.tabs` in your layout extension — that's what collects every `EntityContentBlueprint` registered anywhere in the app (including from third-party plugins), so you don't lose any tabs while still fully owning the UI.